PL1-Estructuras de datos

Dylam Jym Nolasco Vargas 03204543E

Noah Franco Acevedo 49392884Q

Para la práctica hemos usado los TADs Pila y Cola, ambos enlazados.

Colas:

-Recibidos

-Cocina

-Cola ExpedicionNorte

-Cola ExpedicionSur

-Cola ExpedicionEste

-Cola ExpedicionOeste

-Cola PreparacionEspecial

-Cola Finalizados

Pilas:

-Pila CajaNorte

-Pila CajaSur

-Pila CajaEste

-Pila CajaOeste

Hemos dividido el proyecto siguiendo la compilación separada en 5 archivos: Pedido.h y Pedido.cpp (para el modelo de datos), pizzeria.h y pizzeria.cpp (para la declaración e implementación de los TADs Cola y Pila), y main.cpp (para el simulador y menú).

En pizzeria.cpp hemos implementado la lógica de las colas y pilas que usaremos, además de la función generar pedido.

Las colas y pilas son enlazadas y usan memoria dinámica, es decir su tamaño no es fijo, cambia con el estado del programa. Cada pila o cola tiene sus nodos los cuales representan los pedidos. A cada una le hemos implementado las funciones básicas vistas en clase como esVacia(), encolar, desencolar y mostrar...

Funciones destacables:

Generar_pedido (): rellena los campos de un pedido de la siguiente forma:

-id_pedido: va en orden según se vayan generando, controlado por un contador.

-id_cliente: un número aleatorio del 100 al 999

-tipo_pizza: elige aleatoriamente el tipo de pizza

-tamaño: elige aleatoriamente su tamaño

-zona-reparto: elige aleatoriamente su zona de reparto

-personalizada: es un bool cuya probabilidad de true es del 20%

-estado: se inicializa por defecto con el valor del enumerado Recibido.

En pedido.cpp se encuentra el struct pedido con los campos anteriormente mencionados además de los métodos toString() para cada uno de los enum empleados, ya que en c++ estos se muestran por su posición y no por su nombre.

Finalmente, en main.cpp, encontramos todo el bucle principal, la interfaz del programa, las Colas y pilas que se usan en el programa y la lógica de este.

Funciones destacables:

avanzar_N_pedidos(): esta función va avanzando el número que se indique o el máximo que pueda por cola, hacemos el avance al revés, desde la última cola (Expedición) hasta la primera para así evitar que un pedido avance dos etapas. En expedición utilizamos meterEnCaja() para enviarla a su correspondiente caja.

meterEnCaja(): se le pasa por referencia la caja que queremos llenar, en esta función se comprueba si una caja está llena, si lo está se aumenta el contador de cajas llenas de esa zona y se llama a enviarCaja().

enviarCaja(): cuando una caja está llena se usa este método para vaciar toda la caja y poner cada uno de los pedidos en estado de enviado, además de encolarlos en la cola de Finalizados.

retirarPedido(): haciendo uso de una pila auxiliar se va desapilando la caja seleccionada pasándole como parámetro un pedido, al finalizar desapilar p se convierte en ese pedido desapilado, por lo que posteriormente se va comparando el id de ese pedido con el introducido, si son iguales ese pedido pasa a retirado, se encola en Finalizados y deja de buscar coincidencias, si no lo son la pila sigue desapilando hasta que encuentre una coincidencia o no haya más. Al finalizar está función, los elementos que han sido descartados (que se almacenan en la pila auxiliar) son añadidos en la caja (en este momento la caja no tiene los pedidos que están en la auxiliar ni el pedido que se quiere borrar), de este modo borramos el pedido que queríamos.

Los estados en el que un pedido se puede encontrar, que se corresponden a las Colas y Pilas que hemos implementado, son los siguientes:

-Recibido: al crear un pedido su estado inicial es recibido, y se almacena en la cola Recibidos.

-Preparación_especial: pedidos especiales.

-cocina:

-Expedición: cada pedido va a su correspondiente cola de expedición (norte,sur..)

-Caja: cada pedido va a la caja térmica de su zona, están listos para reparto.

-Enviado: Son aquellos pedidos que han sido repartidos.

-Retirado: aquellos pedidos con algún fallo y que no serán repartidos.

El flujo de la pizzería es el siguiente:

Recibido -> Cocina -> Expedición ->Caja -> Finalizados (para pedidos normales)

Recibido -> Preparación Especial -> Cocina -> Expedición -> Caja->Finalizados (para especiales)

En la cola de finalizados están aquellos pedidos con estado enviado o retirado.

La interfaz de este simulador de pizzería tiene 6 opciones:

1-Generar N pedidos:

En esta opción el usuario introduce el número de pedidos que quiere crear, llamando a la función generar_pedido().

2-Mostrar pedidos:

Muestra todas las colas del sistema (Recibidos,Cocina,Preparacion Especial...), además de las cajas térmicas de cada zona enviadas y activas. Incluye el total de cajas enviadas.

3- Avanzar N pedidos un paso:

En esta opción se llama a avanzar_N_pedidos(), la cual avanza cada etapa un paso como máximo con un límite de 10 por cola.

4-Ver caja activa:

Esta opción permite al usuario elegir una caja activa de una zona específica y mostrarla.

5-Retirar pedido de una caja:

El usuario puede retirar un pedido de una caja activa de una zona determinada, indicando el id de ese pedido.

6-Mostrar pedidos finalizados:

Esta última opción permite mostrar aquellos pedidos que fueron retirados en la opción 5 o que fueron enviados.

Problemas encontrados:

-Como hacer que los pedidos avancen un paso solamente: optamos por avanzar en el orden inverso.

-Hacer el menú principal: como mostrar las diferentes opciones planteadas de la forma esperada, usando setw.

-Manejo de char[] en toda la estructura del programa: como copiar char[], como remplazar sus valores, establecerlos...

-Como hacer que al desencolar se sepa cual fue desencolado para poder encolarlo o apilarlo en otro sitio: optamos por hacer que esa función modifique el pedido que se le pasa por referencia dandole el valor del pedido que se desencola, para poder encolarlo en otro sitio.

-generación de datos aleatorios: como utilizar srand().